Welcome!

#0

CS 0447
Introduction to
Computer Programming

Luís Oliveira

Original slides by: Jarrett Billingsley
Modified with bits from: Bruce Childers, David Wilkinson

Fall 2020

What do I need to know now!

<u>The classes will be recorded!</u>

- You will be able to access the videos online
  - They are for your personal use only!
  - Do not distribute them!

- You don't need to turn on your camera
  - If you do, you may be recorded

- You can ask questions via text!
  - Chat is great for that. If I don't stop and read your questions, ask them again
  - But feel free to interrupt me at any point.

Welcome!

- My name is Luis (pronounced Loo-eesh, but I don't really care ☺)
- I'm not from these parts as you can tell from my accent
  - I come from Portugal



i before e except after c…
and this guy's name!

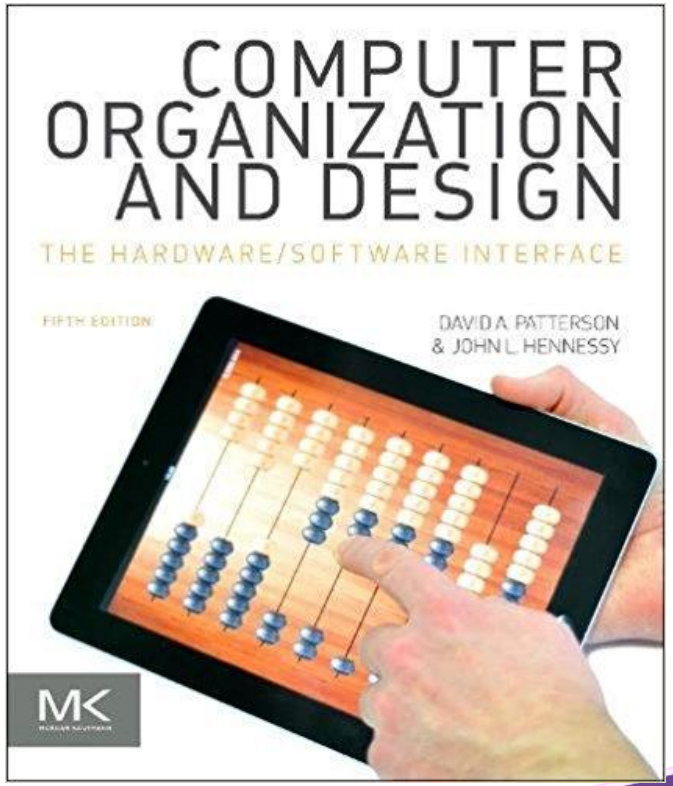
- course site: **`cs.pitt.edu/~loliveira`** or **`luisfnqoliveira.github.io/CS447`**
  - **all the stuff I talk about today is on the course site**
- email: **`loliveira@pitt.edu`**
- office: 5421 SENSQ (haha – more like Zoom) (check site)
- office Hours: TBD (**check the site**)

Textbook

- *Computer Organization & Design* (Patterson & Hennessy)

- Not mandatory!
  - Get it if you really want
  - No readings, no exercises
  - May reference it once in a while

- gets into a lot of detail
  - fun read if you pick this stuff up easily



Grading

- There will be no attendance
  - But you should come to classes!
  - Sometimes the slides alone may not be clear enough
    - Even from seasoned professors! (I know, I've been there!)
- **labs: 20%**
  - there'll be around 12? (~one a week)
  - lowest 2 lab grades are dropped
- **3 projects: 40%**
  - two in MIPS assembly language, one in Logisim
  - **15% two highest grades, 10% lowest grade**
- **2 exams: 40%**
  - 1 midterms, 1 (semi-cumulative) final
  - **Lowest grade 15%; highest is 25%**

Expectations

- Religious absences are excused: contact me ASAP

- Students with disabilities should contact the Office of Disability Resources and Services (DRS) if you haven't already
  - 216 William Pitt Union; **412-648-7890**; TTY:**412-383-7355**

- Please, no comments about sex, gender, race, ethnicity, religion, etc..
  - Anywhere!
  - Just be nice!

- Cheating: **Don't!**
  - 0 on assignment first time,
  - **fail the course** second time.
  - Do not publish your work in public palaces (no **github!**)
  - you can also talk about labs, but still no sharing stuff

DO NOT CHEAT!

- If you're confused, don't cheat, ask me for help?
  - Hot tips for not cheating:
    1. Don’t!
    2. Do not!
  - You have LOTS of resources
    - Me! and the TAs
    - Undergraduate Helpdesk (CRC)

- People can tell when you cheat
  - It is usually quite obvious!
  - So don’t do it, it’s not worth it!
  - The university is quite strict about it.

Teaching

- **No questions are dumb!**
- Are you confused?
  - Surprise, surprise, surprise! IT'S EXPECTED!
  - You are learning a bunch of new stuff!
  - So ASK QUESTIONS!
  - **DON'T STRUGGLE IN SILENCE ON YOUR PROJECTS/LABS!!!!!!!**

- **Regret #1 of my students:**
  - I should have started earlier ;)

- Come to lectures synchronously if you can!
  - You have access to me, we can interact
  - You can ask questions, and get the answers promptly!
  - Please… be interactive ☺

No recitations this first week.

- My suggestion!
  - You take this link, and meet your fellow recitation mates!
    Topic: Recitation get together
    Time: Aug 20, 2020 03:25 PM Eastern Time (US and Canada)
    https://pitt.zoom.us/j/95693091291
    Meeting ID: 956 9309 1291
    Passcode: 447meet
  - Or this link, and meet your fellow recitation mates!
    Topic: Recitation get together
    Time: Aug 21, 2020 12:00 PM Eastern Time (US and Canada)
    https://pitt.zoom.us/j/98887922292
    Meeting ID: 988 8792 2292
    Passcode: 447meet
  - I won't be there, so you can speak freely ☺

FLEXing @ Pitt

IDK you tell me

- Let's discuss some things:
  - What were your experiences last semester with remote teaching?
    - Good/bad/ugly
  - If the university authorizes, do you **want** to attend classes physically?
    - Why? (I really want to know your reasons!)
    - Note the question is not do you **prefer**
      (cause, ya know, we all preferred no CoViD pandemic)
    - **My opinion**: There is no benefit in coming that overweighs your health.
  - Who has timezone restrictions?
    - How should I distribute my office hours?
  - Anything else?

What I know!

- Lectures will be synchronous! And online! ALWAYS!
  - But video will be available!
  - If you can, attend synchronously!
- You can attend lectures asynchronously
  - Have a job? some scheduling problem? Idk?

- Recitations are ALL remote!

- IF we go back to the classroom
  - First lecture, I'll be in the classroom, you will be at home. Why?
    - If technical issues come, I'll just relocate to teach remotely
    - You can experience how it feels (teacher in classroom, students remote)

- EVERYONE WILL FOLLOW THE RULES (distance, mask, etc.!)
  - https://www.coronavirus.pitt.edu/
  - NO EXCEPTION (class will be dismissed)

Introduction and Context

Goals of this course

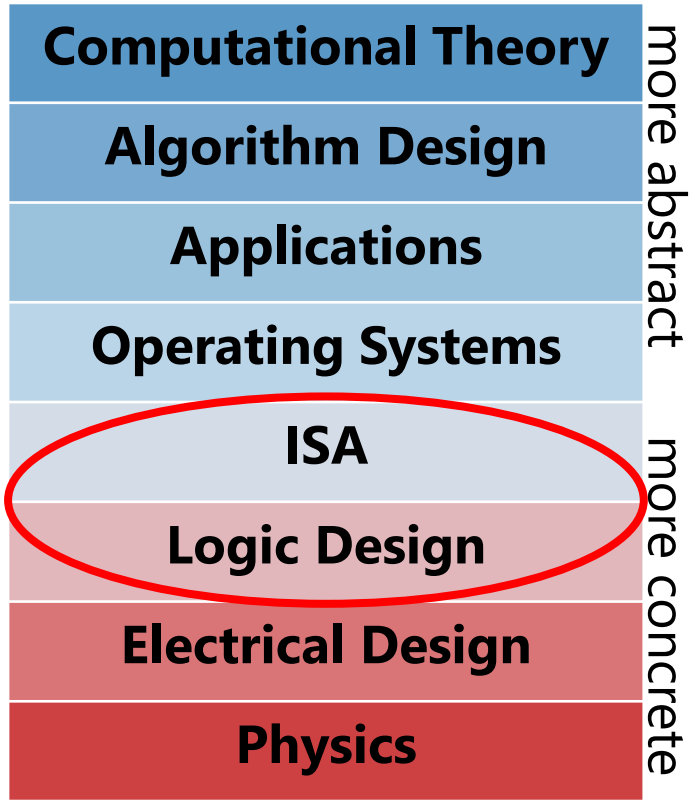
- Why should I take it?
  - Looking under the hood (car analogy)…
  - How do you understand complex modern architectures?
    - Learning the basics (here)

- Topics covered
  - Data representation
  - Assembly language
  - Program execution

- Learning important fundamental skills:
  - Representing numbers in different bases (binary, hexadecimal, …)
  - Logical operations: The binary revolution
  - Logic design: Making circuits without knowing EE :sad:
  - Programming an assembly language – FUN! (your mileage may vary)

Where does this material fit in with CS and EE?

- The “hardware-software interface”
  - where CS and EE overlap
- Each layer affects/is affected by layers above and below
- ISA: Instruction Set Architecture
  - Programmer's **interface** to the computer hardware
- logic design
  - how we make 0’s and 1’s do stuff
  - How do we build a CPU



More goals

- Let's learn Assembly!

- Let's unlearn High-level languages (Java, C)
  - Datatypes/structures? Nope!
  - Infinite number of variables? Nope!

- "Learning assembly is like a car mechanic learning how an engine runs"
  - Normal people don't want to, but it's fun to take things apart!!!
    - Also putting them back together and (hopefully) see them work!
  - And we are not normal people!

FAQ

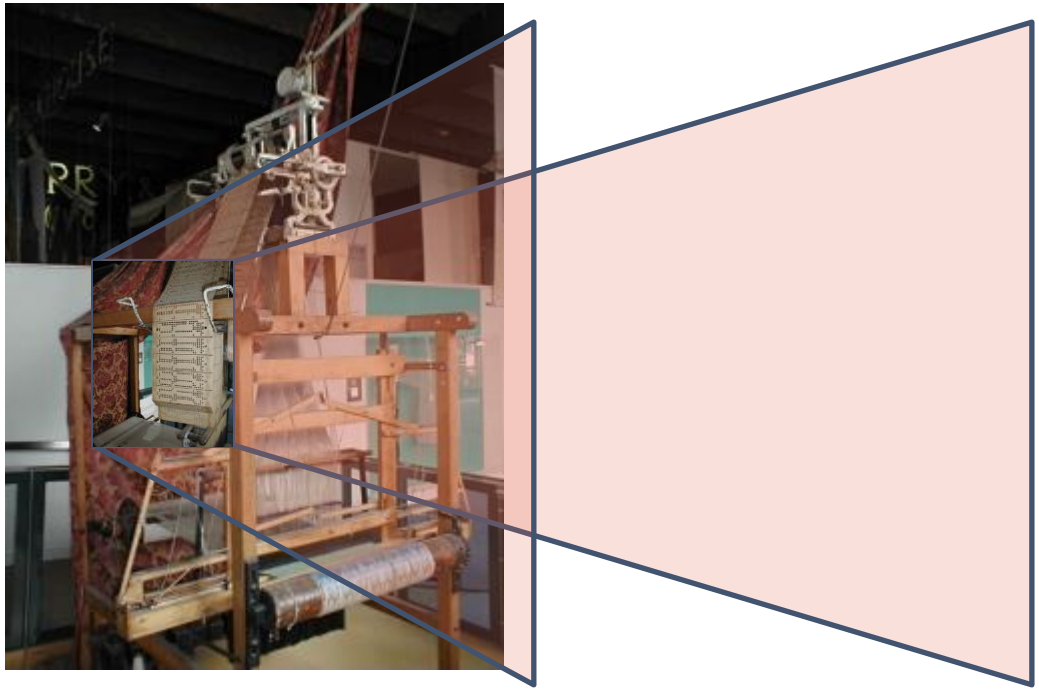
- Maybe the answer for my question is in the next slide. Should I ask?
  - Yes! I'll let you know if that's the case
- I'm sure I should know this. I shouldn't ask!
  - NO! Probably you should not know that yet! So just ASK!
  - NO! Maybe you should know, BUT you don't… So ASK!
- I'm embarrassed I don't want to ask
  - That's fine! But don't be! Others will have the same question! I assure you!
  - But if you want, write the question down, send it to me privately!
  - BUT ASK!!!! (later)
- Should I ask?
  - YES! Interrupt me at any time!!!
  - Like now? INTERRUPT ME!!!

Computers



The Antikythera Mechanism

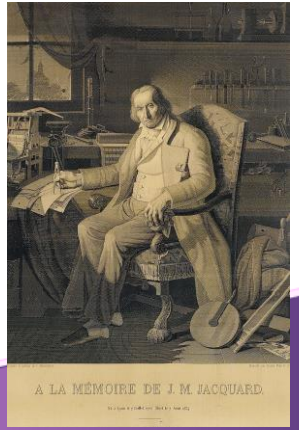
- **Thousands of years old**
  - Late second/early first century BC
  - That's like -100 ish
- **Used for astronomy**
  - Eclipses
  - Astronomical positions

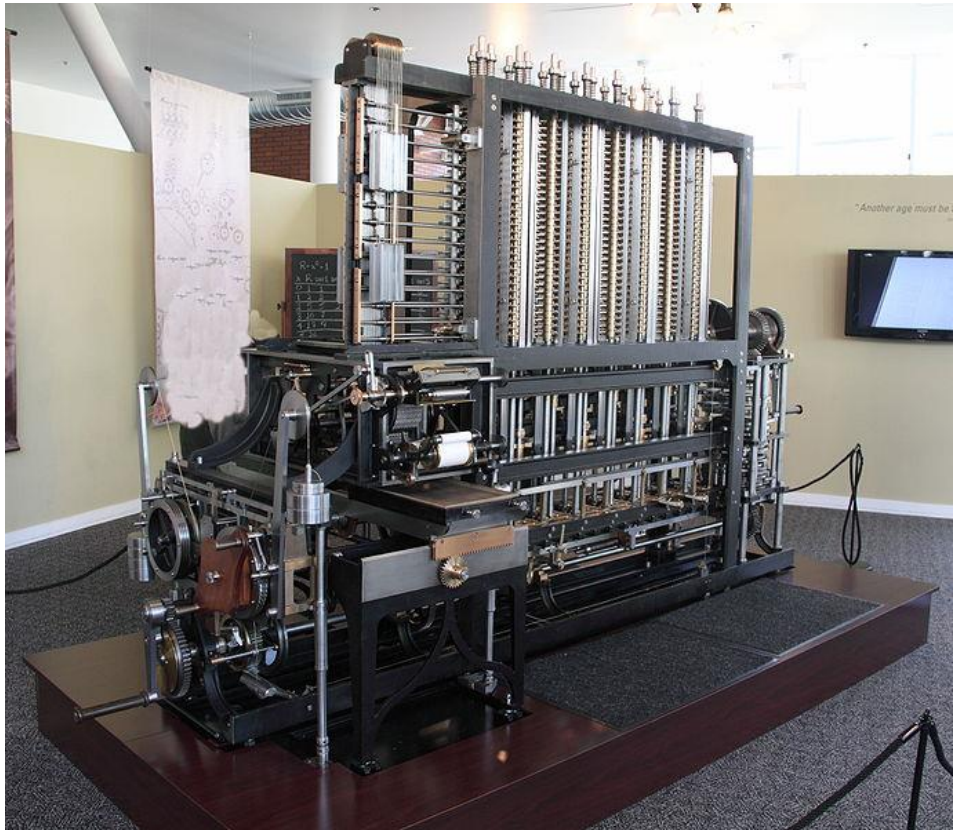


Jacquard machine

- **Mechanical loom (1804)**
  - Programmed using perforated cards
  - Used to produce complex patterns

Woven in silk using 24k punch cards!





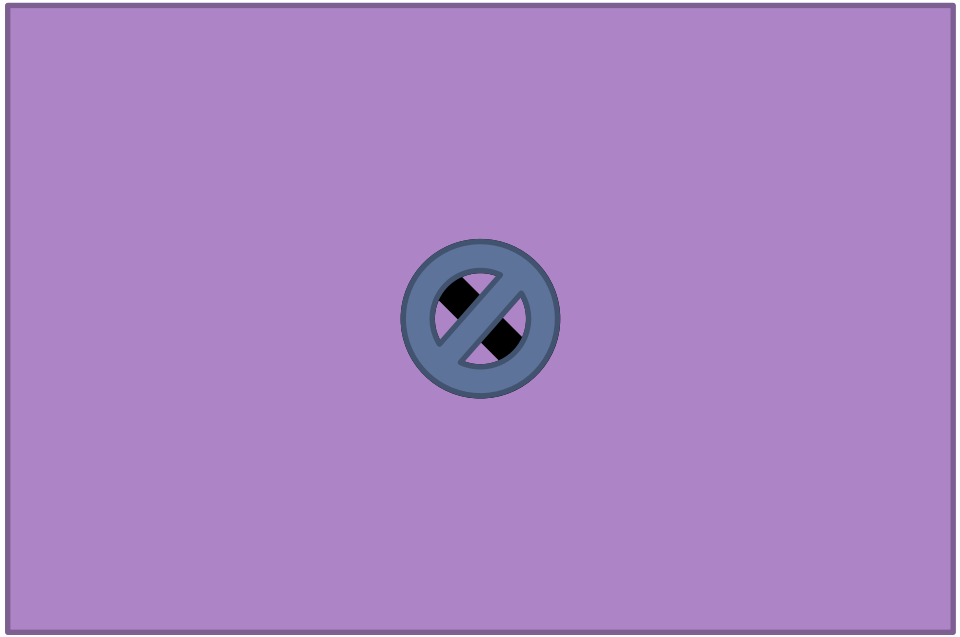
The Differential Engine

- **Designed by Charles Babbage**
  - 1792-1871
- ***Ermmm… Designed… right!***
  - It was intended as a programmable calculator
  - A **multipurpose calculator!**

The pre-history of computers

- The Differential Engine
  - Devised by J.H. Müller in the Hessian army (1784)
  - Designed by Charles Babbage (1819-ish)
  - Built at Science Museum library in London (1980s)
  - Outputs to a table that can be used for printing
    - Copying was a source of error
    - It still is nowadays
    - So never copy results manually if you can avoid it

The Analytical Engine

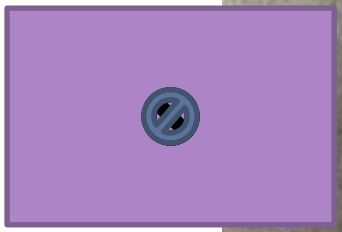


- **Designed by Charles Babbage**
  - *YES! Designed… again!*
- **Mechanical general-purpose computer**
  - Which had many modern characteristics

No actual picture because… it was never built

“The Enchantress of Numbers” - the first programmer

- **Augusta Ada King, Countess of Lovelace**
  - Wrote **algorithms** for this computer →
    - Yeah, that one!
    - But they probably would have worked
  - Translating a paper, she added notes
  - A LOT of notes
    - More than the actual paper
  - Including instructions on how to calculate a number series
    - Note G
  - Studied the relation between maths and music





The Analytical Engine **has no pretensions (…) to *originate* anything**. It **can do whatever we *know how to order it* to perform**.

It **can *follow* analysis**; but it has **no power of *anticipating*** any analytical relations or truths.

(replica)





Hollerith Electric Tabulating System

- **Census happen every 10 years**
  - *Hey, they just did!*
- **It took people 8 years to count responses (in 1880)**
  - It would soon take more than 10!
  - 7,000 cards a day using this system
- **Company would become IBM**
  - After a merge with others

https://www.computerhistory.org/revolution/punched-cards/2/2/5

Bubbles



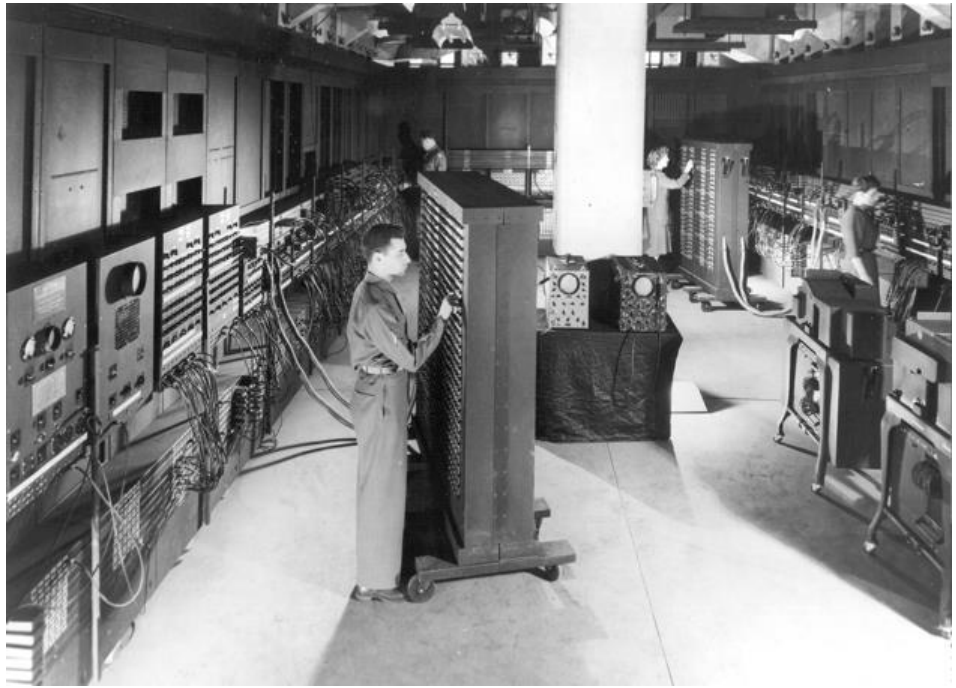
- Check what Bubbles has to say about it ☺
  - https://www.youtube.com/watch?v=L7jAOcc9kBU

Driven by the need for complex calculations

https://computerhistory.org/blog/first-steps-lectures-from-the-dawn-of-computing/

- George Stibitz (Bell Labs)
  - **Day-job**: Electrical engineer
  - Model K – binary addition **with relays** (Boolean algebra)
  - Complex Number Computer – used remotely via telegraph lines!!
  - Art with Amiga (1990s) - http://stibitz.denison.edu/art.html
- Konrad Zuse (Germany)
  - **Day-job**: Aircraft designer (civil engineer)
  - World's first programmable computer
  - Several computers used for military calculations
- John Atanasoff (Iowa State)
  - **Day-job**: Physics professor
  - Built the ABC (Atanasoff-Berry Computer)
    - solved 30 equations in 30 unknowns

ENIAC (Electronic Numerical Integrator and Computer)

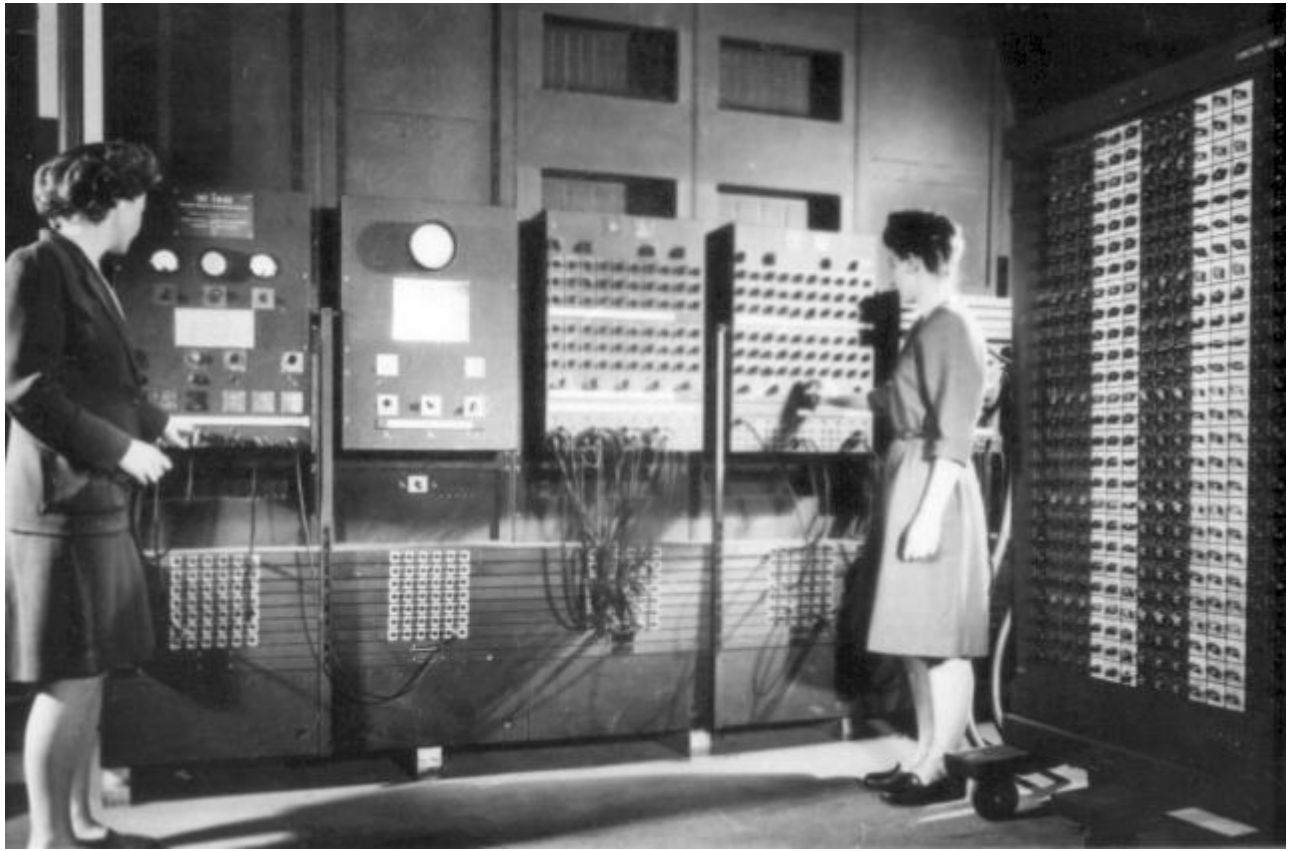


U. S. Army Photo

- 1946 – ENIAC
  - *University of Pennsylvania*
  - *Developed during WWII to calculate balistic missile trajectories*
  - *Designed by :*
    - *John Mauchly*
    - *J. Presper Eckert*
  - *Joined by a huge team!*
  - *Modular and reconfigurable*
    - *Flipping switches and connecting cables*

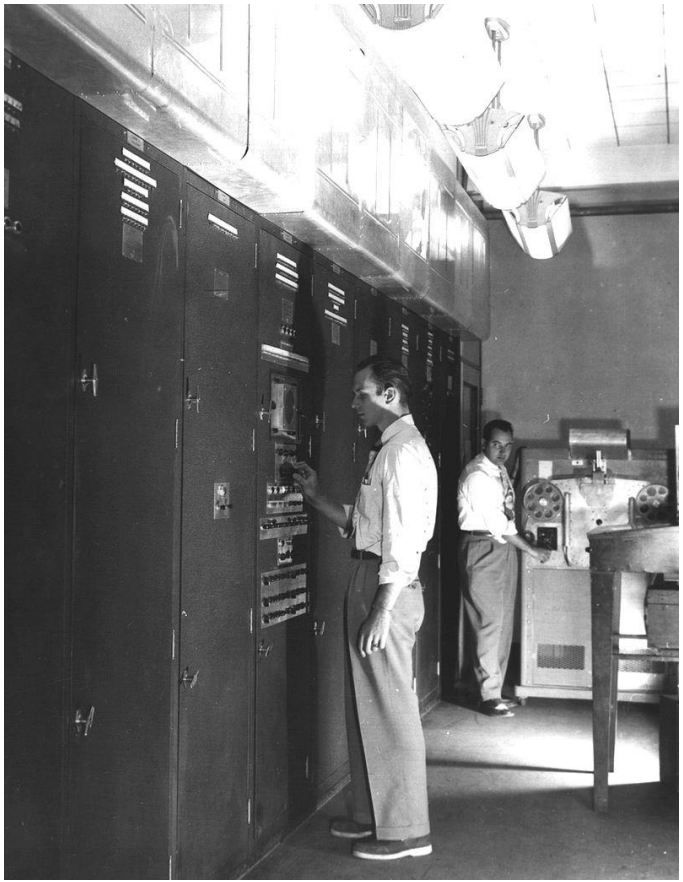
ENIAC (Electronic Numerical Integrator and Computer)

- Some numbers:
  - 18000 valves (tubes)
  - 1500 relays
  - 30 tons
  - 175 kW
  - 5000 additions / s
  - 357 multiplications / s
  - 40 divisions / s
  - Programs "hardwired"



U. S. Army Photo

EDVAC (Electronic Discrete Variable Automatic Computer)

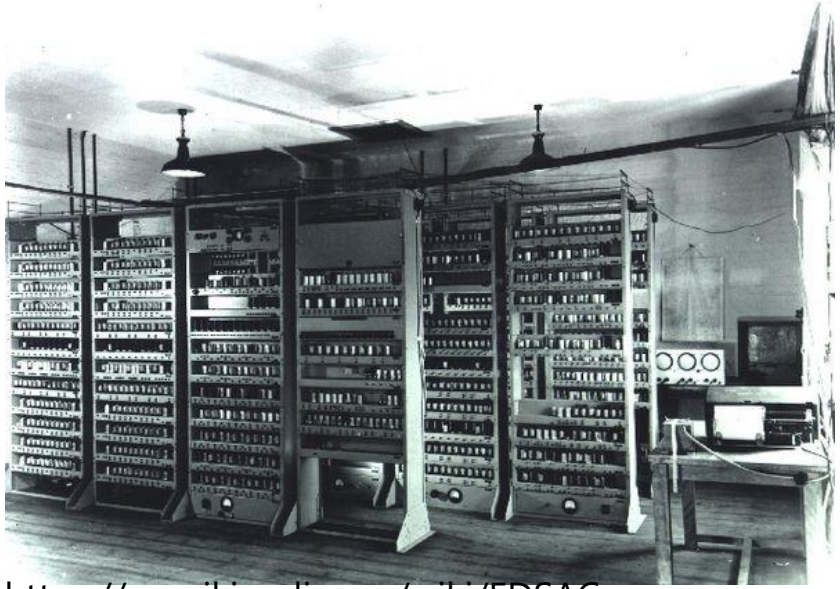


U. S. Army Photo

- 1947 – EDVAC
  - *University of Pennsylvania*
  - *The ENIAC team joined by John Von Neumann*
  - *A computer with a new concept:*
    - *"Memory Stored Program" – same as data*
  - *Became operational in 1951*



EDSAC (Electronic Delay Storage Automatic Calculator)



https://en.wikipedia.org/wiki/EDSAC

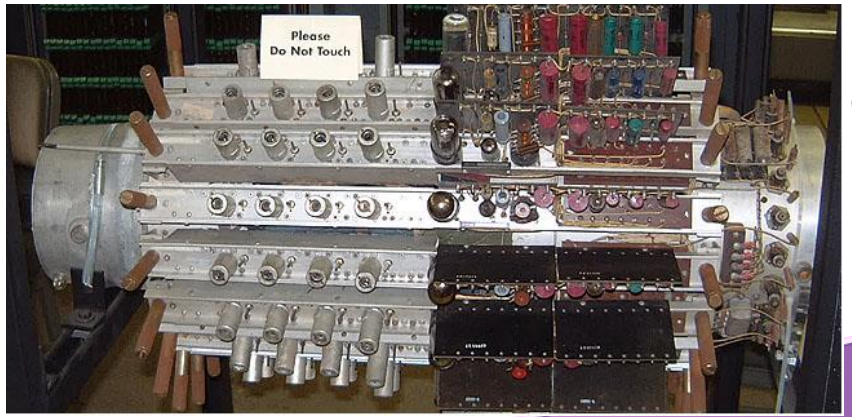
- 1949 – EDSAC
  - *Cambridge University*
  - *Designed by Maurice Wilkes*
  - *Based on the first EDVAC draft*
    - Not to be better, but to be used!
      - accessible and practical vs. push technology
    - Was completed before the EDVAC!
  - Used for scientific research
    - Chemistry, Medicine, Physics

https://www.tnmoc.org/edsac

UNIVAC (Universal Automatic Computer)



- 1951 – UNIVAC
  - *First commercial computer!*
    - *Sold 46! Units*
    - *Used to predict the 1952 presidential election*
  - *Used MERCURY!! memory (as did the EDSAC)*



Delay Storage

https://en.wikipedia.org/wiki/Delay_line_memory

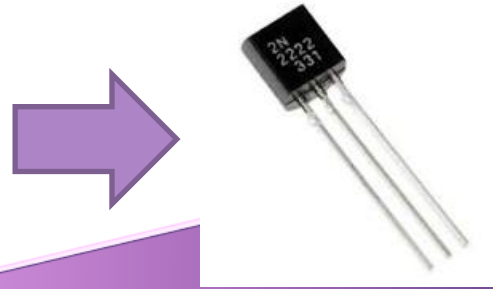
Then came the transistor



https://en.wikipedia.org/wiki/Transistor

- The symbol for a transistor
  - *Photo taken in the university where I did my master degree*
- They were tiny
  - *Didn't get HOT!*
  - *Didn't break as often*

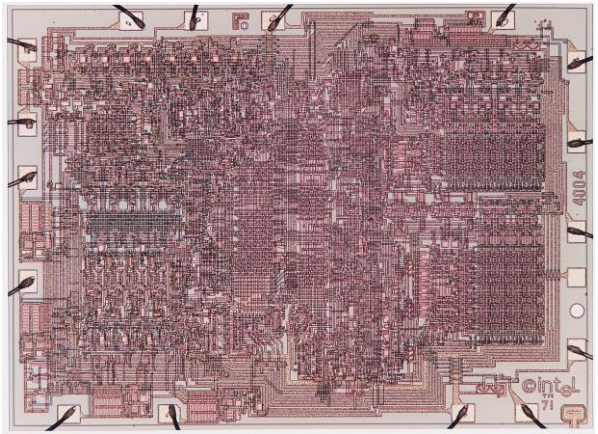




Then came the Integrated circuit

2300 of these in there

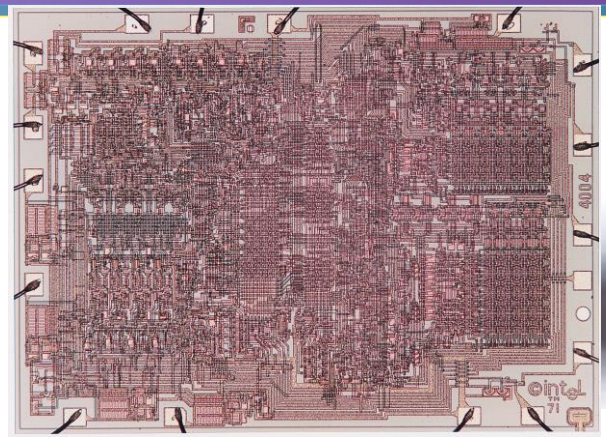


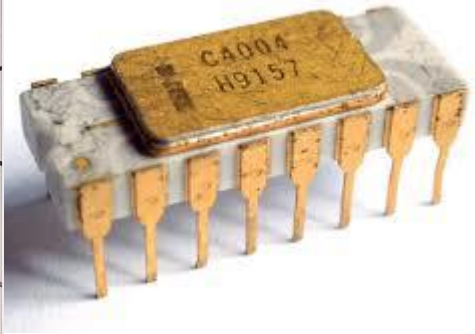


- **Things became** tiny
  - *More transistors could be fitted*
  - *Cheaper circuits*
  - *More affordable*

Extremely brief story of Intel CPUs

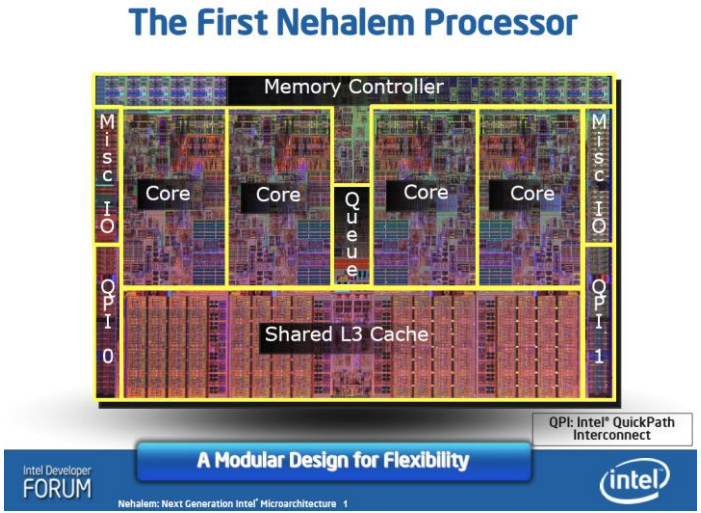
- 1971 – Intel 4004
  - 4-bit microprocessor
  - with 2300! Transistors





- 2004 – Pentium 4
  - x86 32-bit
  - 125 Million transistors

- 2017 – Kaby Lake
  - x86_64 64-bit
  - >1000 Million! (undisclosed?)



Moore's Law

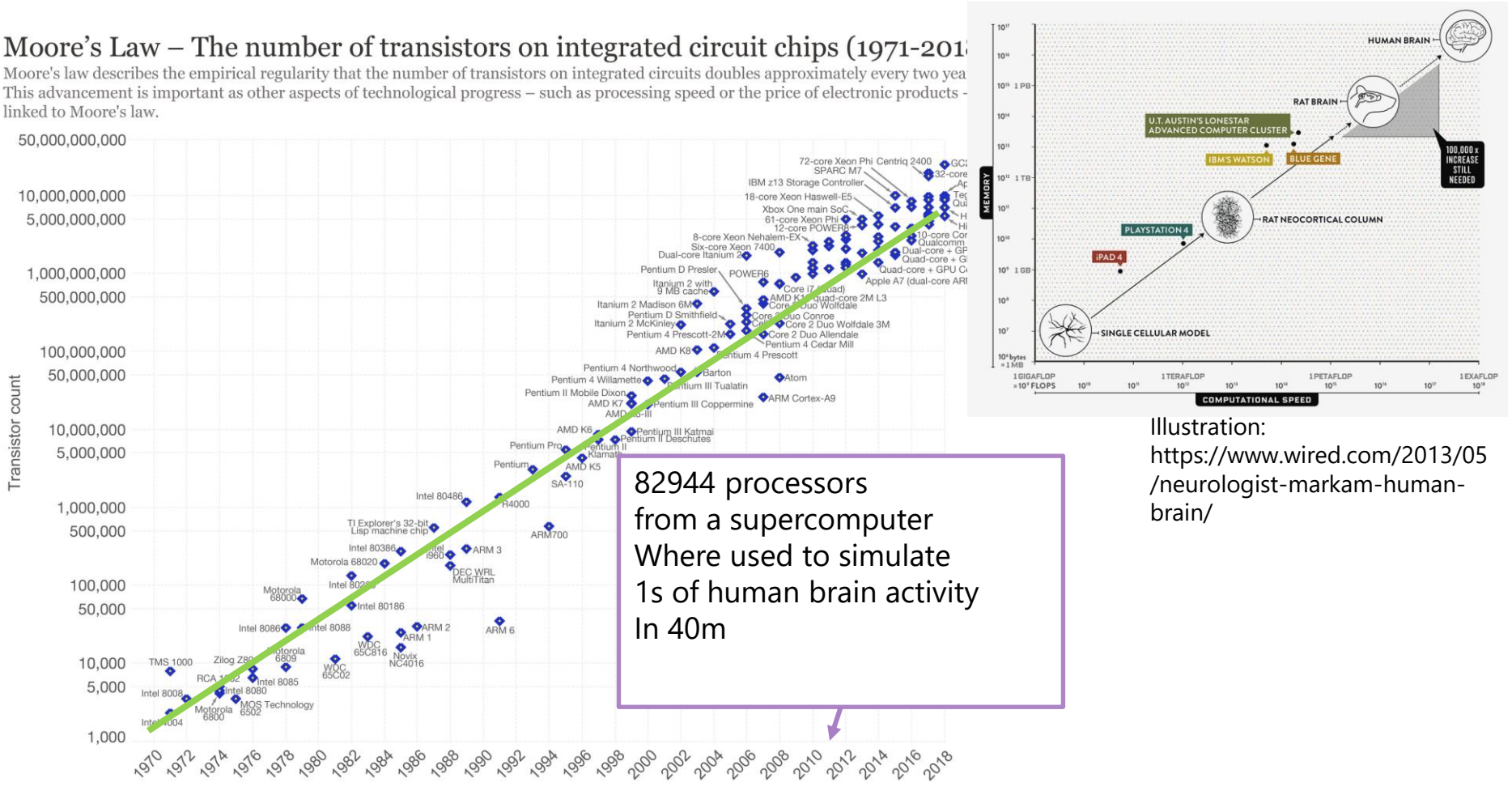


Illustration: https://www.wired.com/2013/05/neurologist-markam-human-brain/

Data source: Wikipedia (https://en.wikipedia.org/wiki/Transistor_count)
The data visualization is available at OurWorldinData.org. There you find more visualizations and research on this topic.

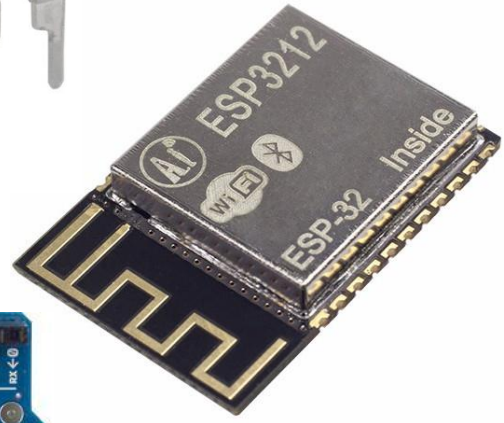
Licensed under CC-BY-SA by the author Max Roser.

All different but all (mostly) the same

Classes of computers: Embedded (Microcontrollers)

- 8/16-bit architectures are still common
  - What does this mean?
- As little as 32 BYTES of memory!
  - almost always run one, built-in program
- Focus on ultra-low power, cost, and size
  - becoming more common every day
  - “Internet of Things” (IoT)
  - now your fridge can crash,
    your TV can crash,
    your dishwasher can crash,
    everything can crash!

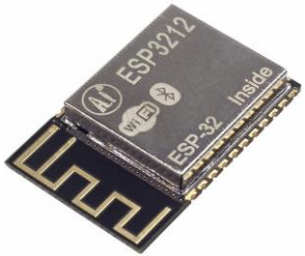






Some are small and cheap





- They run a single program
  - *E.g your refrigerator*
- Are used by hobbyists
  - *For small projects*

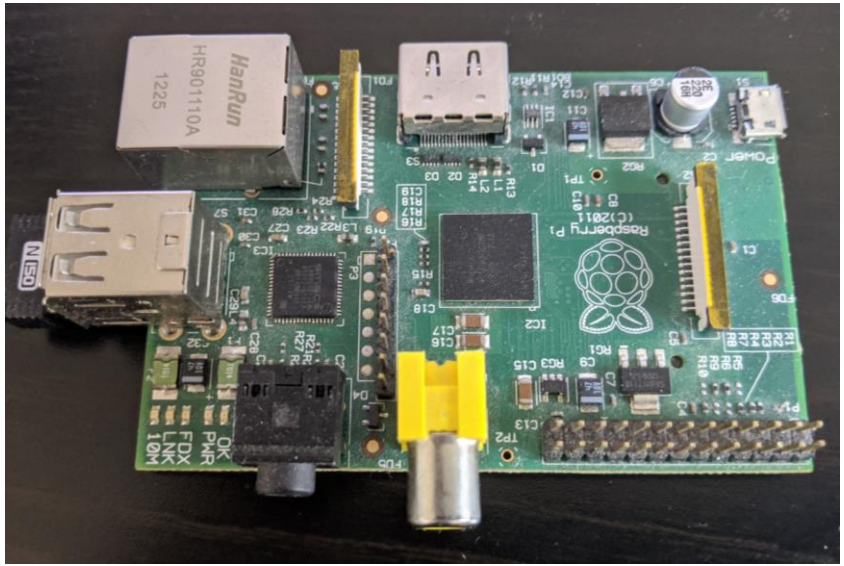
Classes of computers: Consumer-grade (PC/Mobile)

- 1-8 cores, 32/64-bit architectures, MB-GB of memory, GB-TB of persistent storage
- **multitasking** operating systems
- similar *capabilities*, different design *goals*
- focus is real-time user interaction: productivity, media, browsing, games
- Energy consumption is still relevant
  - Mobile



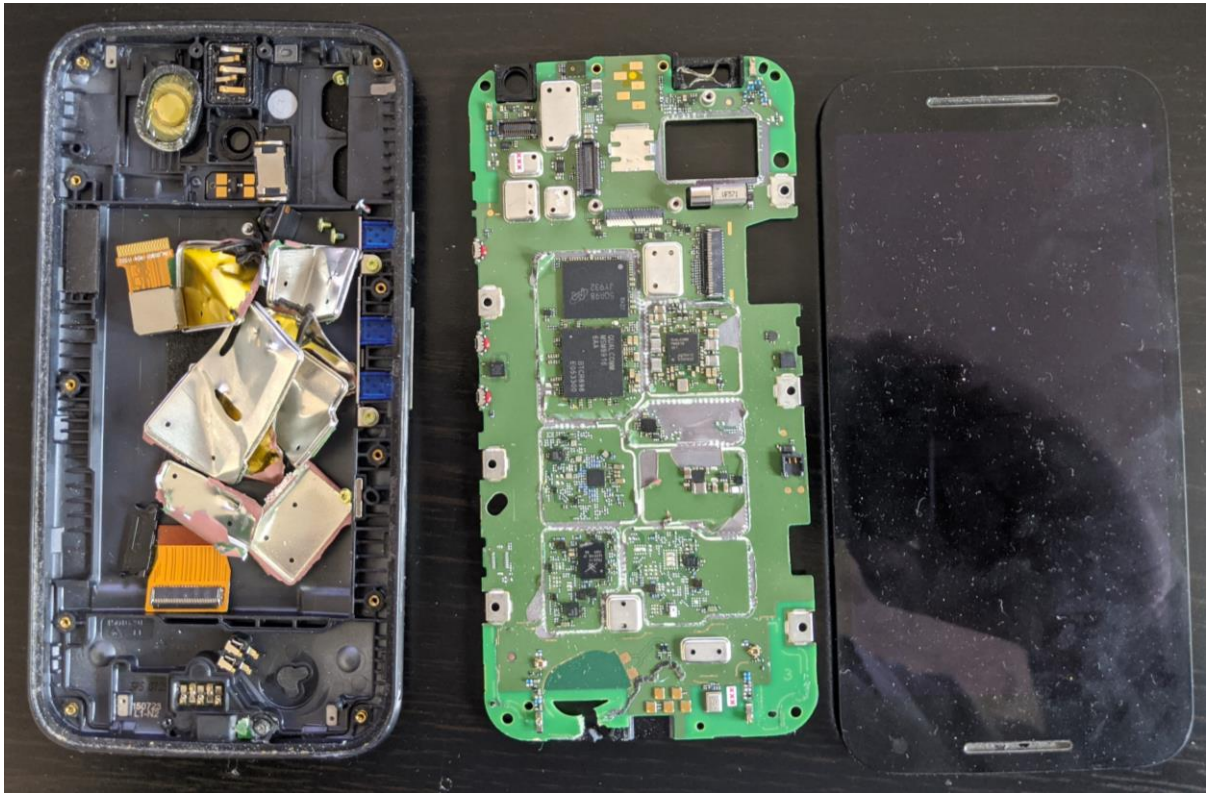


Some are small, cheap, and powerful



- The Raspberry Pi
  - *Affordable, yet powerful*
  - *~$35*
  - *Can be used for A LOT of projects*
    - *Home automation*
    - *Affordable PC*
    - *Great to learn how to program on a budget*

Some are small, expensive, and powerful



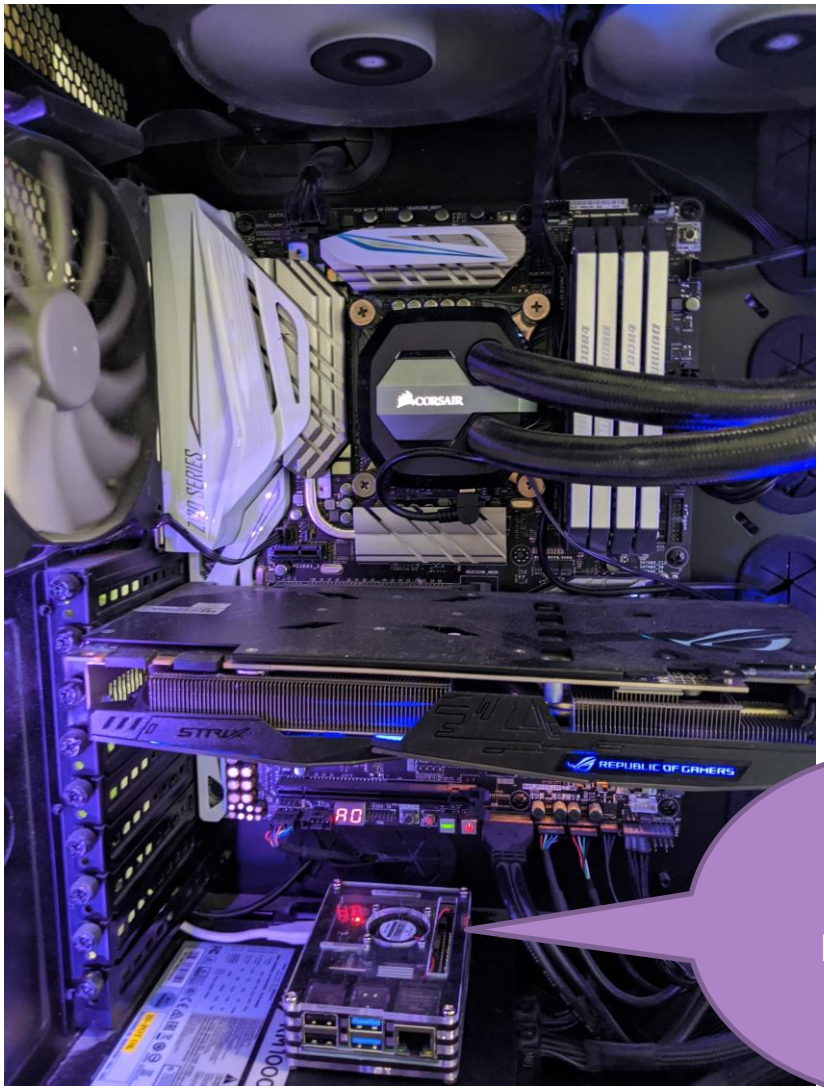
- My Moto G3 ☹
  - *I sacrificed it for you!*
    - *(battery was bloated, it had to go)*
  - *Had to rip some parts :D*
  - *Mobility is important (~1 day)*
  - *Portability is important*
  - *But it runs beefy apps!*

We hold them in our laps (does anyone do that often?)



- Power and mobility
  - Battery life is important
    - We want to fly with them ☹
  - Weight is important
  - Run demanding programs!

We have them at home



My computer

- Desktop computers
  - *Wide range of prices ($300 to +$5k)*
  - *Energy consumption not important*
    - *Beyond cost and heat generation*
  - *Performance*
    - *Games*
    - *Browsers!!*
    - *Word?*

Classes of computers: Servers and Mainframes

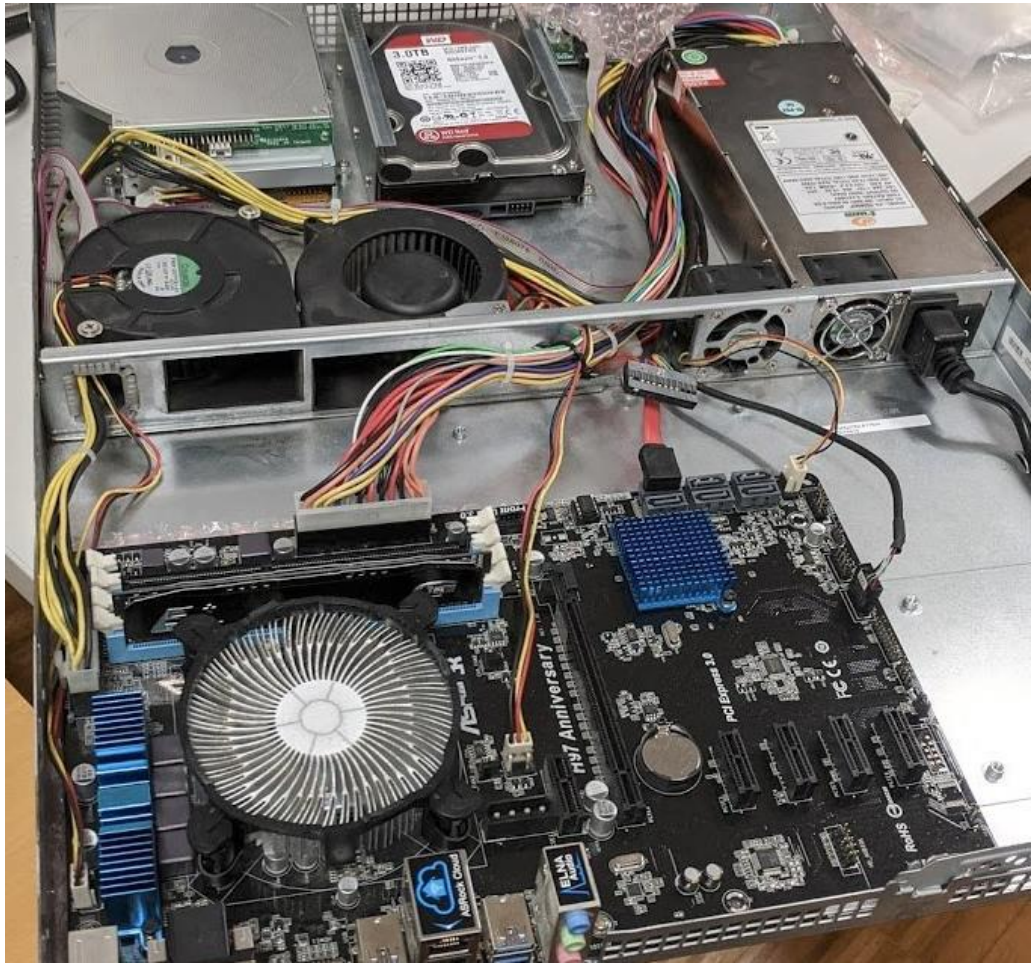
- from high end desktops to much more powerful machines or groups of machines
- dozens of cores, 32GB+ of RAM, massive storage systems, d networking
- focus on **real-time** data delivery - either from storage or a
- redundancy and hot-swappability
- goal is 100% uptime
- power and cooling become *huge* concerns







We have them in warehouses



- Server on a “drawer” (rack)
  - *Don’t have monitors*
    - *People don’t “use them” directly*
    - *Non-interactive*
  - *Crunch numbers and return results*
    - *Webpages*
    - *Remote storage (e.g., box)*

Classes of computers: Supercomputers

- cluster of hundreds to thousands of CPUs
- focus on crunching ENORMOUS datasets **non-interactively**
- science, research, design, and simulation
  - and now, stock trading and "cryptocurrencies"...



All looking pretty much the same











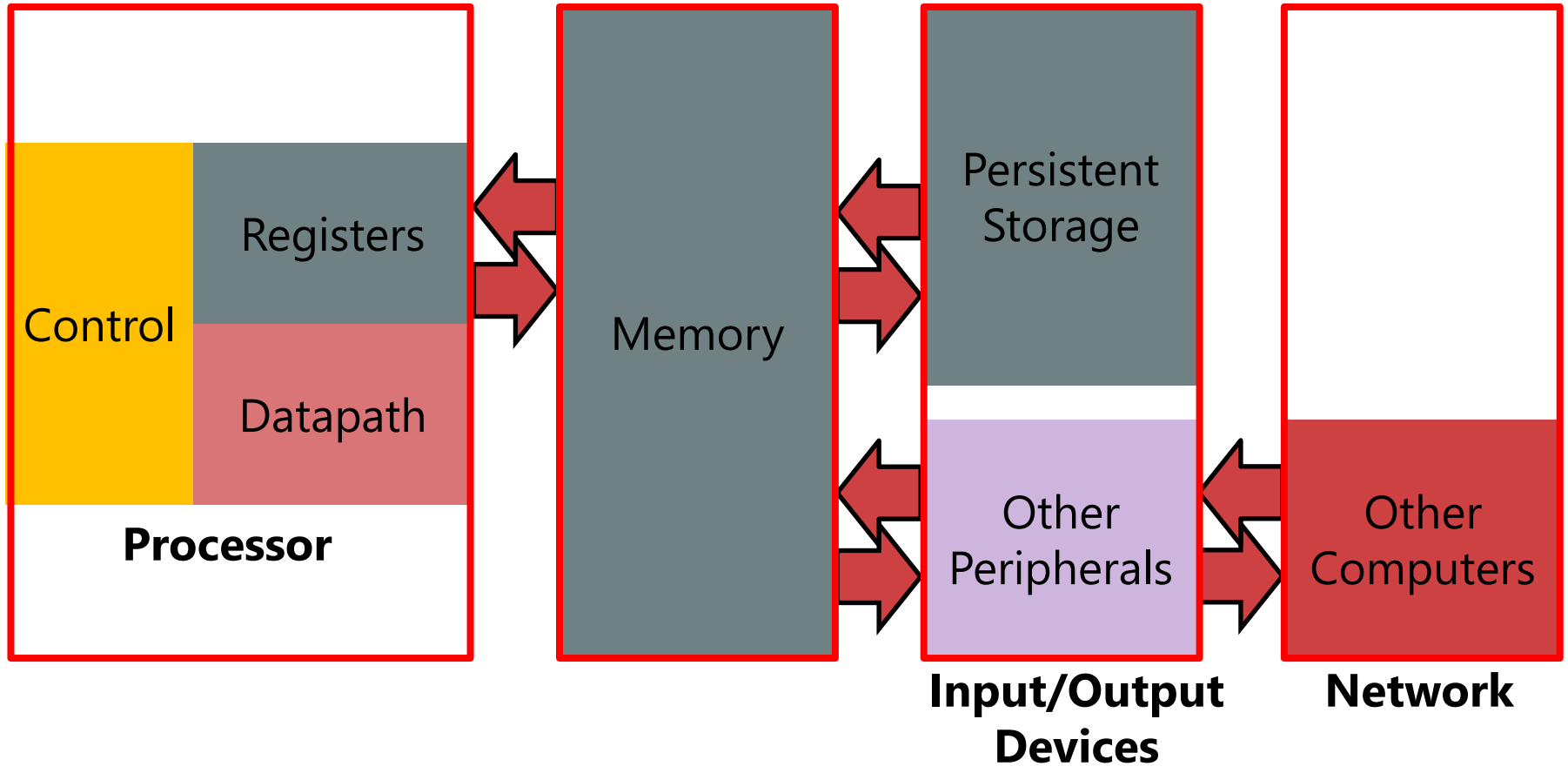






The Internet

General computer organization



Assembly

Different language levels



```
x=x+1
```

High-level language
Written by humans → Abstracted

```
lw   $t0, x
addi $t0, $t0, 1
sw   $t0, x
```

Assembly language
Written by humans→ No abstraction

```
10011101100110011001111101111001
11011110110010111011101001111001
10011100100110110001101111111011
```

Random numbers ☺



Machine language
Spoken by the CPU→ Binary

Easier to read

Closer to the hardware

So what exactly is assembly?

- **Assembly**: Human-readable representation of machine code
- **Machine code**: Machine code is what a CPU actually runs

- Essentially it's the "atoms" that make up a program
  - CPUs are actually pretty simple in concept
    - (oh wait, we still have a whole semester about it… huh)
- Each CPU has its own machine language and therefore its own assembly language
- We'll be using **MIPS:**
  - Not that common (why then?)
  - Yet, often found in surprising places. (Nintendo 64, PS1/2, FPGAs)
  - Very influential, and most common assembly looks like it.
    - ARM and RISC-V are similar-ish ISA seeing much more usage

Different language levels



```
x=x+1
```

High-level language
→ Same for different CPUs

```
lea x, %eax
mov 0(%eax), %ecx
inc    %ecx
mov %ecx, 0(%eax)
```

```
lw   $t0, x
addi $t0, $t0, 1
sw   $t0, x
```



Assembly language
→ Different for different CPUs




```
01001111011110010001 1
01100101110011110110 0
10111001101011010011 1
10111100100011011001 0
1110
```

Random numbers ☺

```
10011101100110011 00
11110111100111011 1
10110010111011010 0
11110011001110010 1
10110001101111111 01
```

Random numbers ☺



Machine language
→ Different for different CPUs

IS ASSEMBLY LANGUAGE USEFUL TODAY?

- Short answer: ***YES***
- Assembly is "fast", so we should use it for everything!

  --- **NO!!!** ---
- No type-checking, no control structures, very few abstractions
  - Probably you can be fairly fast using compiler optimizations

    --- **Fairly impractical for large things** ---
- Tied to a particular CPU.
  - So, large programs have to be rewritten (usually) to work on new things.
- Yet: good for specialized stuff.
  - Critical paths and "boot" code in Kernels / Operating Systems
  - HPC (simulators, supercomputer stuff)
  - Real-time programs (video games; tho increasingly less / abstracted away)
  - And…

Embedded systems

- You'd be **amazed** at *how many* consumer devices have CPUs.
- Many are programmed largely/entirely in assembly (or C)



and next time we'll start

- Data representations
- How does a CPU look at numbers, letters, etc.
- Learn about number bases:
  - Decimal (that's what we use!)
  - Binary
  - Octal
  - Hexadecimal